software dynamic. For us at Twilio, AI isn't just about writing software faster; it's about liberating access and turning static user interactions into rich, real-time conversations.

Q: Do you think it is an incremental boost, or is it a total reset of the developer stack?

A: It's neither a total reset nor just an incremental boost—it's a massive leap forward. The real shift is where control and leverage sit. Developers can no longer skip writing specs, tests, or system designs. Because AI handles so much raw execution, thinking through system architecture ahead of time is more critical than ever.

Q: Given that you have worked primarily with languages like Go, Node.js, and Rust, how do you keep these core backend skills sharp when high-level AI tools handle so much boilerplate code today?

A: At the end of the day, accountability still sits with the engineer who submits the pull request. I use GenAI to fan out and explore multiple approaches to a problem quickly, but then I fan in and do the due diligence to actually understand the output. Whenever an LLM takes an unexpected approach, I pause, read up on the algorithm, and learn from it. Far from making developers dull, it's helped me stay sharp in areas I might not have explored otherwise.

Q: People often confuse developer relations (DevRel) with marketing or sales. How do you define what a Developer Evangelist does, and why is this role so important for engineering platforms?

A: While Twilio sells to businesses, over 10 million developers are actually integrating our APIs. They need to know how Twilio fits into their broader stack, and that's where DevRel comes in. We act as the connecting glue: we provide the technical depth behind marketing campaigns, run hands-on workshops to help sales convert opportunities, and bring raw feedback back to product teams. It's an exploratory, educational role sitting right at the intersection of enterprise software and the developer community.

Q: You have advocated a lot for crafting great developer experiences. What makes a tool truly great versus just functional, and how do you spot friction in the developer workflow?

A: The ecosystem effect makes all the difference. A functional tool executes a task; a great tool is welcoming, easy to apply, and supported by a strong community, clear docs, and peripheral tooling like SDKs or MCP servers. Spotting friction requires active listening. We don't just stand at event booths; we talk to builders about their pain points—which is how the feedback led us to completely redesign the Twilio console.

Q: How do developers across the Asia-Pacific and Japan (APJ) region differ from Western developers when building communication workflows—and how do their specific needs around multi-language support and early testing shape Twilio's regional product strategy?

A: Two main things set APJ developers apart: language diversity and a hands-on testing culture. In markets like India, Singapore, and Southeast Asia, English-only tech falls flat. Whether building public-facing AI for banks or voice agents for local consumers, developers prioritise multi-language support—in Mandarin, Hindi, Tamil, or Bahasa—from day one. Both Twilio's communication channels and the underlying LLMs must accurately parse and respond in these languages. Additionally, APJ engineers like to build and test early. Based directly on feedback from India, Singapore, and Australia, we recently revamped our trial access programs so regional developers can get hands-on and prototype right out of the gate.

Q: How do you view the open source model of software development from a DevRel perspective?

A: Open source is here to stay because a strong technical project naturally brings people together. My biggest influence here is Linus Torvalds and the Linux kernel's approach: the technology itself births the community that builds it, not the other way around. Trying to rally a community without a solid core product usually fails. Whether it's an open source tool or a startup chasing product-market fit, you need a clear cause, roadmap, and focused maintainers. Beyond that, open source instills incredible engineering discipline. Because all contributions are subject to public peer review, developers hold themselves to a higher standard and naturally avoid pushing sloppy code.

Q: Have you been contributing to any open source projects personally?

A: Over the last few years, my direct contributions have mostly been to my employers' public projects. I closely follow developments in ecosystems like Node.js and Rust. While at Twilio, my contributions have been focused within our ecosystem, and prior to this, at Freshworks, I contributed significantly to the Freshworks CLI, which was a public project. The challenge is always the same: how to listen to users and prioritise what is important to the community versus what is important to the company. Commercial open source is an interesting category because you must balance responsibilities to both the community and the organisation.